

Phase 7

Encryption Technical Reference

RSA + AES: the equations, the process, the files, and the full code

This document contains everything for Phase 7 in one place:

- The RSA equations — key generation, encryption, decryption — with a worked example.
- The AES equations — XOR and CTR-mode keystream.
- The complete encrypted-call process, step by step.
- Every file in the crypto module and exactly what each one does.
- The full source code of the crypto module, explained piece by piece.

Builds on: Phase 6 (plain voice between two devices).

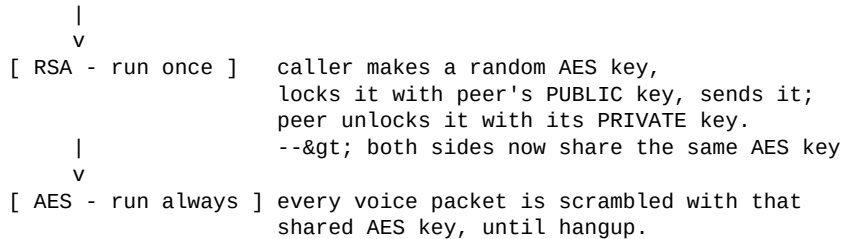
Goal of Phase 7: the voice on the LTE link becomes unintelligible to anyone who intercepts it.

1. The Plan in One Picture

Two ciphers are used together. Each fixes the other's weakness.

	RSA-512	AES-128
Type	Asymmetric (public + private key)	Symmetric (one shared key)
Speed	Slow	Fast
Used	ONCE, at call setup	On EVERY audio packet
Job	Deliver the AES key safely	Scramble the actual voice

CALL CONNECTS



2. RSA — The Equations

2.1 Key generation (done once on the PC by keygen.py)

Two large random prime numbers **p** and **q** are chosen. From them:

$$N = p \times q$$

N is the *modulus*. For RSA-512 it is a 512-bit number (64 bytes). **N** is part of both the public and the private key.

$$\text{phi}(N) = (p - 1) \times (q - 1)$$

phi(N) (Euler's totient) is a helper number used only during key generation. It is secret and discarded afterwards.

$$\text{choose } E \text{ such that } 1 < E < \text{phi}(N) \text{ and } \text{gcd}(E, \text{phi}(N)) = 1$$

E is the *public exponent*. In practice **E** = 65537 is used for every key. It must share no common factor with **phi(N)**.

$$D = E^{-1} \bmod \text{phi}(N)$$

D is the *private exponent*. It is the modular inverse of **E**: the number for which (**E** × **D**) divided by **phi(N)** leaves remainder 1. **D** is the secret that makes decryption possible.

Result — the two keys:

Key	Made of	Secret?
Public key	(N , E)	No — shared with anyone
Private key	(N , D)	Yes — never leaves the device

2.2 Encryption and decryption

The message **m** must be a number smaller than **N**. To encrypt it into the ciphertext **c**:

$$c = m^E \bmod N$$

To decrypt the ciphertext back to the message:

$$m = c^D \bmod N$$

"mod N" means: divide by N and keep only the remainder. Encryption uses the public pair (N, E); decryption uses the private pair (N, D). Locking with E can only be undone by D, and D cannot be found without the original primes p and q.

2.3 Worked mini-example (tiny numbers, to see it actually work)

Step	Equation	Result
Pick primes	p, q	p = 11, q = 13
Modulus	$N = p \times q$	N = 143
Totient	$\phi = (p-1)(q-1)$	$\phi = 10 \times 12 = 120$
Public exponent	choose E	E = 7
Private exponent	$D = E^{-1} \bmod \phi$	D = 103 (7 x 103 = 721 = 6x120 + 1)
Encrypt m = 9	$c = m^E \bmod N$	$c = 9^7 \bmod 143 = 48$
Decrypt c = 48	$m = c^D \bmod N$	$m = 48^{103} \bmod 143 = 9$ (original back)

Public key here = (143, 7); private key = (143, 103). The number 48 is what travels the network — useless without D = 103.

2.4 The 64-byte block and padding

RSA-512 always works on a fixed 64-byte block. The AES key is only 16 bytes, so the other 48 bytes are filled with random padding before encryption. Padding does two jobs: it makes the message m large (close to N, as RSA needs) and it makes every encryption different even if the same key is sent twice. The receiver strips the padding after decrypting.

2.5 Why RSA is secure

An attacker sees N, E and the ciphertext c. To recover m they need D. D can only be computed from $\phi(N)$, and $\phi(N)$ can only be computed from the primes p and q. The attacker only has $N = p \times q$. Splitting a large N back into p and q ("factoring") has no fast method known — for a 512-bit N it is computationally enormous. That gap is the entire security of RSA.

RSA-512 is weak by modern standards and would never protect a real system. It is used here only because it is light enough for the STM32F401 and is perfectly adequate to demonstrate the principle for an academic project.

3. AES — The Equations

3.1 XOR — the reversible operation

AES encryption is built on XOR. XOR compares two bits: equal bits give 0, different bits give 1.

$$0 \oplus 0 = 0 \quad 0 \oplus 1 = 1 \quad 1 \oplus 0 = 1 \quad 1 \oplus 1 = 0$$

Its key property — applying the same value twice cancels out:

$$(P \oplus K) \oplus K = P$$

So if you XOR the audio with a secret value to scramble it, XOR-ing the result with the *same* secret value brings the audio back.

3.2 CTR mode — turning AES into a keystream

AES itself encrypts one fixed 16-byte block. CTR (Counter) mode uses AES to generate an endless stream of random-looking bytes — the *keystream* — by encrypting a simple counter:

$$S_i = \text{AES}(K, \text{counter}_i)$$

K is the 16-byte session key; counter_i is the block number (0, 1, 2, ...). Each S_i is one 16-byte block of keystream. The audio is then encrypted and decrypted by XOR:

$$C = P \oplus S_i \text{ (encrypt)}$$

$$P = C \oplus S_i \text{ (decrypt)}$$

P = plaintext audio, C = ciphertext (scrambled) audio. Because both devices hold the same key K and use the same counter, both generate the same keystream S_i , so the receiver's XOR exactly undoes the sender's XOR.

3.3 The counter

The counter must advance by one for every 16-byte block and must never repeat for a given key — reusing a counter value would leak information. Both sides start at 0 when the call begins and count up together. Encrypt and decrypt are the same operation, which is why a single function `aes_ctr_crypt()` serves both directions.

4. The Encrypted Call — Full Process

Steps 1-7 are the Phase 6 call. Phase 7 inserts one new step, 5.5, between the call connecting and the audio starting.

```

Device A          relay          Device B
-----          -
REG 001 -----&gt;
                REG 002 &lt;-----
[A presses CALL]
CALL 002 -----&gt; INC 001 -----&gt;
                [B presses ANSWER]
                &lt;----- ANS
                &lt;----- GO
===== STEP 5.5: KEY EXCHANGE (RSA) =====
A: make random 16-byte AES key -&gt; pad to 64 bytes
A: blob = rsa_encrypt(padded) (locked with B's PUBLIC key)
A: send blob -----&gt;
                B: rsa_decrypt(blob) with B's PRIVATE key
                B: strip padding -&gt; recovers the AES key
--&gt; both devices now hold the SAME 16-byte AES key
===== STEP 6: ENCRYPTED AUDIO =====
A: audio -&gt; aes_ctr_crypt() -&gt; scrambled -&gt; LTE -----&gt; B: aes_ctr_crypt() -&gt; audio
B: audio -&gt; aes_ctr_crypt() -&gt; scrambled -&gt; LTE -----&gt; A: aes_ctr_crypt() -&gt; audio
...continues until...
[HANGUP] -&gt; AES key is erased; next call generates a fresh one

```

Step	What happens	Crypto function
1-5	Register, CALL, INC, ANS, GO (Phase 6 call setup)	none
5.5a	Caller generates a random 16-byte AES key	crypto_init / RNG
5.5b	Caller pads it to 64 bytes and RSA-locks it with peer public key	rsa_encrypt()
5.5c	The 64-byte blob is sent as one control message over LTE	tcp_send_line
5.5d	Peer RSA-unlocks the blob with its own private key, strips padding	rsa_decrypt()
5.5e	Both sides load the recovered key into AES	aes_ctr_set_key()
6	Every audio packet is scrambled before send, unscrambled after receive	aes_ctr_crypt()
7	Hangup — the AES key is erased from RAM	none

5. The Files — What Each One Does

5.1 New files added for Phase 7

File	Where	What it does
keygen.py	MicDAC/ (runs on PC)	Generates the RSA keypairs once. Writes crypto_keys.h. Never runs on the STM32.
crypto_keys.h	MicDAC/Core/Inc/	Pure data: the RSA key numbers as byte arrays. No logic. Produced by keygen.py.
crypto.h	MicDAC/Core/Inc/	The header: declares the crypto functions so main.c can call them.
crypto.c	MicDAC/Core/Src/	The encryption logic: RSA bignum math + AES-CTR. The "encryption tool".
aes.c / aes.h	MicDAC/Core/Src/ + Inc/	The AES block cipher core (tiny-AES-c, a small public-domain library).

5.2 Existing files and their roles

File	Role
Core/Src/main.c	The program: call state machine, buttons, LTE, audio. Calls into crypto.c.
Core/Src/adc.c, tim.c, i2c.c, usart.c, dma.c, gpio.c	CubeMX hardware setup (ADC, timers, I2C DAC, modem UART, DMA, pins).
CMakeLists.txt	Build recipe. Must be edited to add crypto.c and aes.c.
Server/relay.py	Runs on the laptop. Passes messages and audio between the two devices.

5.3 How the files depend on each other

```

keygen.py  --(run once on PC)-->  crypto_keys.h
                                   |
                                   | #include
                                   v
main.c  --calls-->  crypto.h  <--declares--  crypto.c  --uses-->  aes.c
  |                                     |
  | (the call logic)                   | (RSA bignum + AES-CTR)
  v                                     v
adc/tim/i2c/usart/dma/gpio.c          crypto_keys.h (the keys)

CMakeLists.txt  --tells the compiler-->  build main.c + crypto.c + aes.c + ...

```

6. The Code

6.1 crypto.h — the interface main.c sees

```
#ifndef CRYPTO_H
#define CRYPTO_H
#include <stdint.h>

/* One-time setup at startup. */
void crypto_init(void);

/* ---- RSA-512: works on fixed 64-byte blocks ---- */
/* Lock a 64-byte block with the PEER's public key. */
void rsa_encrypt(const uint8_t in[64], uint8_t out[64]);
/* Unlock a 64-byte block with MY private key. */
void rsa_decrypt(const uint8_t in[64], uint8_t out[64]);

/* ---- AES-128 in CTR mode ---- */
/* Load the 16-byte session key for this call. */
void aes_ctr_set_key(const uint8_t key[16]);
/* Encrypt OR decrypt len bytes in place. block0 is the
   starting counter for this buffer. */
void aes_ctr_crypt(uint8_t *buf, uint32_t len, uint32_t block0);

#endif /* CRYPTO_H */
```

Just five function declarations. main.c includes this header and calls these names — it does not need to know the math inside.

6.2 crypto_keys.h — the keys (made by keygen.py)

```
#ifndef CRYPTO_KEYS_H
#define CRYPTO_KEYS_H
#include <stdint.h>

#define IS_DEVICE_A 1 /* 1 on device ...001, 0 on device ...002 */

static const uint8_t RSA_A_N[64] = { 0x.., 0x.., ... }; /* device A modulus */
static const uint8_t RSA_A_D[64] = { 0x.., 0x.., ... }; /* device A private */
static const uint8_t RSA_B_N[64] = { 0x.., 0x.., ... }; /* device B modulus */
static const uint8_t RSA_B_D[64] = { 0x.., 0x.., ... }; /* device B private */
static const uint32_t RSA_E = 65537u; /* shared public exp */

#if IS_DEVICE_A
#define MY_N RSA_A_N
#define MY_D RSA_A_D
#define PEER_N RSA_B_N
#else
#define MY_N RSA_B_N
#define MY_D RSA_B_D
#define PEER_N RSA_A_N
#endif
#endif
```

This file is only data. The 0x.. bytes are filled in by keygen.py. Both devices use the same file; only IS_DEVICE_A differs.

6.3 crypto.c — RSA: representing 512-bit numbers

The STM32 has no 512-bit integer type. A big number is stored as an array of 16 small 32-bit pieces ("limbs"): 16 x 32 = 512 bits. limb[0] is the least significant.

```
#include "crypto.h"
#include "crypto_keys.h"
#include "aes.h"
#include <string.h>

#define LIMBS 16 /* 16 x 32-bit = 512-bit numbers */
typedef uint32_t bn_t[LIMBS]; /* a big number; bn[0] = least significant */

/* The RSA keys are stored big-endian (64 bytes). Convert to a limb array. */
static void bytes_to_bn(const uint8_t b[64], bn_t out) {
```

```

    for (int i = 0; i < LIMBS; i++) {
        int j = (LIMBS - 1 - i) * 4;
        out[i] = (((uint32_t)b[j] << 24) | ((uint32_t)b[j+1] << 16) |
                  ((uint32_t)b[j+2] << 8) | ((uint32_t)b[j+3]));
    }
}
static void bn_to_bytes(const bn_t in, uint8_t b[64]) {
    for (int i = 0; i < LIMBS; i++) {
        int j = (LIMBS - 1 - i) * 4;
        b[j] = (uint8_t)(in[i] >> 24); b[j+1] = (uint8_t)(in[i] >> 16);
        b[j+2] = (uint8_t)(in[i] >> 8); b[j+3] = (uint8_t)(in[i]);
    }
}

```

6.4 crypto.c — RSA: add, subtract, compare

```

/* Compare: returns -1 if a<b, 0 if equal, +1 if a>b. */
static int bn_cmp(const bn_t a, const bn_t b) {
    for (int i = LIMBS - 1; i >= 0; i--) {
        if (a[i] < b[i]) return -1;
        if (a[i] > b[i]) return 1;
    }
    return 0;
}
/* r = a + b, returns the carry out of the top limb. */
static uint32_t bn_add(bn_t r, const bn_t a, const bn_t b) {
    uint64_t carry = 0;
    for (int i = 0; i < LIMBS; i++) {
        uint64_t s = (uint64_t)a[i] + b[i] + carry;
        r[i] = (uint32_t)s;
        carry = s >> 32;
    }
    return (uint32_t)carry;
}
/* r = a - b (assumes a >= b). */
static void bn_sub(bn_t r, const bn_t a, const bn_t b) {
    uint64_t borrow = 0;
    for (int i = 0; i < LIMBS; i++) {
        uint64_t d = (uint64_t)a[i] - b[i] - borrow;
        r[i] = (uint32_t)d;
        borrow = (d >> 32) & 1;
    }
}

```

6.5 crypto.c — RSA: modular multiply and modular power

These two functions ARE the RSA equations $c = m^E \bmod N$ translated into code that works on 512-bit numbers.

```

/* r = (r + add) mod m, given r < m and add < m. One subtraction is enough
   because the sum is always less than 2m. */
static void bn_addmod(bn_t r, const bn_t add, const bn_t m) {
    uint32_t carry = bn_add(r, r, add);
    if (carry || bn_cmp(r, m) >= 0) bn_sub(r, r, m);
}

/* r = (a * b) mod m -- double-and-add, bit by bit through b.
   No 1024-bit product is ever formed; only 512-bit add/sub are used. */
static void bn_modmul(bn_t r, const bn_t a, const bn_t b, const bn_t m) {
    bn_t acc; memset(acc, 0, sizeof acc); /* acc = 0 */
    for (int i = LIMBS - 1; i >= 0; i--) {
        for (int bit = 31; bit >= 0; bit--) {
            bn_addmod(acc, acc, m); /* acc = 2*acc mod m */
            if ((b[i] >> bit) & 1)
                bn_addmod(acc, a, m); /* acc = acc + a mod m */
        }
    }
    memcpy(r, acc, sizeof acc);
}

/* r = (base ^ exp) mod m -- square-and-multiply, bit by bit through exp.
   This is the RSA core: c = m^E mod N and m = c^D mod N. */
static void bn_modexp(bn_t r, const bn_t base, const bn_t exp, const bn_t m) {
    bn_t acc; memset(acc, 0, sizeof acc); acc[0] = 1; /* acc = 1 */

```

```

    for (int i = LIMBS - 1; i >= 0; i--) {
        for (int bit = 31; bit >= 0; bit--) {
            bn_modmul(acc, acc, acc, m);          /* square      */
            if ((exp[i] >> bit) & 1u)             /* multiply    */
                bn_modmul(acc, acc, base, m);
        }
    }
    memcpy(r, acc, sizeof acc);
}

```

Square-and-multiply: to compute base^{exp} , walk the bits of the exponent from top to bottom. At each bit, square the running value; if the bit is 1, also multiply by base. This computes a 512-bit power in only ~512 steps instead of an impossible number of multiplications.

6.6 crypto.c — RSA: the public encrypt / decrypt functions

```

/* Lock a 64-byte block with the PEER's public key: out = in ^ E mod PEER_N */
void rsa_encrypt(const uint8_t in[64], uint8_t out[64]) {
    bn_t m, n, e, c;
    bytes_to_bn(in, m);
    bytes_to_bn(PEER_N, n);
    memset(e, 0, sizeof e); e[0] = RSA_E;      /* small public exponent -> bignum */
    bn_modexp(c, m, e, n);                     /* c = m ^ E mod N */
    bn_to_bytes(c, out);
}

/* Unlock a 64-byte block with MY private key: out = in ^ D mod MY_N */
void rsa_decrypt(const uint8_t in[64], uint8_t out[64]) {
    bn_t c, n, d, m;
    bytes_to_bn(in, c);
    bytes_to_bn(MY_N, n);
    bytes_to_bn(MY_D, d);
    bn_modexp(m, c, d, n);                     /* m = c ^ D mod N */
    bn_to_bytes(m, out);
}

```

Notice how short these are: all the work is in `bn_modexp`. `rsa_encrypt` is literally the equation $c = m^E \bmod N$; `rsa_decrypt` is $m = c^D \bmod N$.

6.7 crypto.c — AES-CTR

The AES block cipher itself comes from `tiny-AES-c` (`aes.c` / `aes.h`). This code wraps it in CTR mode: encrypt a counter to get keystream, then XOR.

```

static uint8_t aes_key[16];

void aes_ctr_set_key(const uint8_t key[16]) {
    memcpy(aes_key, key, 16);                  /* store the session key */
}

/* Encrypt OR decrypt buf[] in place (CTR mode is symmetric: same operation). */
void aes_ctr_crypt(uint8_t *buf, uint32_t len, uint32_t block0) {
    struct AES_ctx ctx;
    AES_init_ctx(&ctx, aes_key);
    for (uint32_t off = 0; off < len; off += 16) {
        uint8_t ks[16] = {0};
        uint32_t ctr = block0 + off / 16;      /* this block's counter */
        ks[12] = (uint8_t)(ctr >> 24); ks[13] = (uint8_t)(ctr >> 16);
        ks[14] = (uint8_t)(ctr >> 8); ks[15] = (uint8_t)(ctr);
        AES_ECB_encrypt(&ctx, ks);             /* ks = AES(counter) = Si */
        uint32_t n = (len - off < 16) ? (len - off) : 16;
        for (uint32_t i = 0; i < n; i++)
            buf[off + i] ^= ks[i];             /* C = P XOR Si */
    }
}

void crypto_init(void) {
    /* nothing needed yet; placeholder for RNG seeding if required */
}

```

This single function is used by the sender (audio in -> scrambled out) and by the receiver (scrambled in -> audio out). XOR with the keystream is its own inverse, so no separate decrypt function is needed.

6.8 Hooking the crypto into main.c

main.c includes crypto.h and calls the functions at two points: once at call setup, and around every audio packet.

```
#include "crypto.h"

/* --- at call setup, once the call connects (after GO) --- */
uint8_t aes_key[16];
uint8_t blob[64];
/* caller side: */
make_random_bytes(aes_key, 16);          /* fresh session key */
build_padded_block(plain64, aes_key);    /* 16-byte key -> 64-byte padded */
rsa_encrypt(plain64, blob);              /* lock with peer public key */
/* ...send blob over LTE as one message... */
aes_ctr_set_key(aes_key);

/* peer side: */
/* ...receive blob over LTE... */
rsa_decrypt(blob, plain64);              /* unlock with my private key */
extract_key(aes_key, plain64);           /* strip padding */
aes_ctr_set_key(aes_key);

/* --- around audio (in the ST_IN_CALL block) --- */
/* outgoing, just before tcp_send_audio(out, HALF_SIZE): */
aes_ctr_crypt(out, HALF_SIZE, tx_counter);
tx_counter += HALF_SIZE / 16;

/* incoming, before writing the bytes into play_buf: */
aes_ctr_crypt(in, n, rx_counter);
rx_counter += n / 16;
```

6.9 CMakeLists.txt — build the new files

The compiler must be told about the new source files. Add crypto.c and aes.c to the source list in CMakeLists.txt:

```
# in the add_executable(...) / target_sources(...) source list:
Core/Src/crypto.c
Core/Src/aes.c
```

7. Summary

Item	In one line
$N = p \times q$	The RSA modulus, built from two secret primes.
$\phi = (p-1)(q-1)$	Helper number for key generation; secret, then discarded.
E	Public exponent (65537). Part of the public key.
$D = E^{-1} \bmod \phi$	Private exponent. The secret that allows decryption.
$c = m^E \bmod N$	RSA encryption — lock with the public key.
$m = c^D \bmod N$	RSA decryption — unlock with the private key.
$S_i = \text{AES}(K, \text{counter})$	AES-CTR keystream block.
$C = P \text{ XOR } S_i$	AES encrypt and decrypt (same operation).
keygen.py	PC tool: generates the RSA keys, writes crypto_keys.h.
crypto_keys.h	Holds the RSA key numbers. Data only.
crypto.h	Declares the 5 crypto functions for main.c.
crypto.c	The encryption tool: RSA bignum math + AES-CTR.
aes.c / aes.h	The AES block cipher core (tiny-AES-c library).
rsa_encrypt / rsa_decrypt	Lock / unlock a 64-byte block. Used once per call.
aes_ctr_crypt	Scramble / unscramble an audio packet. Used every packet.

The whole idea, one paragraph: RSA solves "how do two devices agree on a secret over an open line" — the caller locks a fresh AES key with the peer's public key, only the peer's private key can open it. That happens once. Then AES, using that shared key, scrambles every voice packet fast enough for real-time audio, until the call ends.

End of Phase 7 — Encryption Technical Reference.